Abstract

La clase inició el tema 9, centrado en cómo evaluar si un agente o sistema multiagente es realmente fiable y está preparado para funcionar en un entorno empresarial.

En las clases anteriores se había trabajado en el diseño, la implementación y la integración de agentes con sistemas empresariales. En esta sesión, el enfoque cambió hacia la medición de su rendimiento: comprobar si resuelven correctamente las tareas, si mantienen resultados consistentes, cuánto tardan en responder, qué disponibilidad tienen y cuánto cuesta cada operación.

La idea principal fue que un agente que funciona durante una demostración no necesariamente está preparado para producción. Para llevarlo a un entorno real se necesitan pruebas repetibles, métricas objetivas y criterios que permitan justificar las decisiones técnicas y económicas.

La clase utilizó como ejemplo la tienda virtual ficticia Lumenshop, que ya se había trabajado en sesiones anteriores. A partir de ella se diseñaron pruebas para evaluar el comportamiento del agente, incluyendo consultas normales, situaciones límite y preguntas malintencionadas.

Resumen general

Del desarrollo a la evaluación

En el tema anterior se había estudiado cómo diseñar e implementar agentes y conectarlos con herramientas empresariales. Ahora se aborda la etapa posterior: evaluar lo construido.

Este proceso no termina con una primera versión funcional. Los resultados de las pruebas pueden indicar que es necesario modificar la arquitectura, cambiar el modelo o ajustar las instrucciones. Después de cada modificación, se debe volver a medir para comprobar si realmente hubo una mejora.

Este documento ha sido generado con inteligencia artificial.

La evaluación permite pasar de las impresiones subjetivas a resultados medibles que puedan presentarse ante responsables técnicos y de negocio.

¿Qué significa que un agente sea fiable?

Un agente fiable no es simplemente uno que consigue resolver una consulta. También debe hacerlo de manera consistente, respetando los límites para los que fue diseñado.

Por ejemplo, si un agente de atención al cliente responde correctamente una vez, pero falla en varias ejecuciones posteriores con la misma pregunta, no se puede considerar suficientemente fiable.

Por eso es necesario ejecutar pruebas repetidas y recopilar métricas que permitan conocer su comportamiento general. Esto es especialmente importante porque los modelos de IA son no deterministas: una misma consulta puede producir resultados diferentes.

Las cuatro familias principales de métricas

Durante la clase se explicaron cuatro grupos de métricas para evaluar los agentes:

- Calidad: mide si el agente resuelve correctamente la tarea. Por ejemplo, si entrega el dato solicitado o utiliza la herramienta adecuada.

- Fiabilidad: mide la consistencia de los resultados entre diferentes ejecuciones. Permite saber cuántas veces consigue responder correctamente.

- Latencia: mide cuánto tiempo tarda el sistema en responder al usuario.

- Disponibilidad: mide con qué frecuencia el sistema está operativo y puede atender las solicitudes.

Además, se mencionó el coste como una métrica fundamental para determinar si el sistema es viable económicamente.

Métricas Pass@K y Pass^K

Se presentaron dos métricas relacionadas con la fiabilidad y el comportamiento no determinista de los modelos:

- Pass@K: indica si el sistema consigue resolver correctamente una tarea al menos una vez dentro de un número determinado de intentos.

- Pass^K: indica si el sistema consigue resolver correctamente todos los intentos realizados.

La diferencia es importante. Pass@K permite conocer si existe alguna posibilidad de obtener una respuesta correcta, mientras que Pass^K exige consistencia en todas las ejecuciones.

En atención al cliente, el profesor destacó la importancia de Pass^K, porque cada interacción representa una oportunidad real de atender correctamente a una persona. No es suficiente con que el agente acierte en algunas ocasiones.

Este documento ha sido generado con inteligencia artificial.

Latencia y percentil 95

La latencia representa el tiempo que debe esperar el usuario para recibir una respuesta. No basta con calcular el tiempo promedio, porque algunas solicitudes pueden tardar considerablemente más que otras.

Por eso se explicó el uso de dos medidas:

- Mediana: representa el tiempo central de las respuestas.

- Percentil 95 (P95): representa el tiempo por debajo del cual se completan el 95 % de las solicitudes. Permite observar las esperas más largas y detectar situaciones en las que el usuario podría experimentar demoras.

En el ejemplo de Lumenshop se planteó como referencia que las respuestas del chat deberían tardar menos de 10 segundos.

Estas mediciones también ayudan a decidir si es necesario cambiar de modelo, modificar la arquitectura o implementar un sistema de respaldo (fallback) para cuando el modelo principal no responda.

Disponibilidad y coste por tarea

La disponibilidad mide cuántas veces el sistema puede responder correctamente desde el punto de vista técnico. Cuando no está disponible, es necesario identificar si el problema proviene de la arquitectura desarrollada o de un proveedor externo, como el servicio del modelo.

Por otro lado, el coste por tarea permite calcular cuánto dinero y cuántos tokens se consumen al resolver una solicitud.

Esta información es importante para comparar el sistema de IA con otras alternativas, como mantener un equipo humano de atención al cliente. También permite determinar si el presupuesto asignado al proyecto es suficiente.

Métricas operativas y tasa de escalado humano

Además de las métricas generales, se explicaron indicadores relacionados directamente con el funcionamiento empresarial:

- Tiempo máximo de respuesta.

- Porcentaje de disponibilidad.

- Coste por tarea resuelta.

- Porcentaje de solicitudes que necesitan intervención humana.

La tasa de escalado mide cuántos casos terminan siendo derivados a una persona. El profesor explicó que no necesariamente debe ser cero: si el sistema nunca escala un caso, podría significar que no está identificando correctamente las situaciones que requieren ayuda humana.

Tampoco sería conveniente que una proporción demasiado alta de solicitudes terminara en manos de una persona, porque eso podría indicar que el agente no está resolviendo suficientes tareas por sí mismo.

Este documento ha sido generado con inteligencia artificial.

El objetivo es encontrar un equilibrio, teniendo en cuenta los criterios y necesidades del negocio.

Diseño de conjuntos de pruebas

Para evaluar el agente de Lumenshop se preparó un conjunto de casos de prueba almacenados en un archivo JSON.

Se definieron tres categorías:

- Consultas normales: preguntas habituales que el agente debería resolver sin dificultades.

- Casos límite o borde: situaciones menos frecuentes o más específicas que podrían provocar errores.

- Casos adversarios: preguntas diseñadas intencionalmente para intentar manipular el agente, saltarse sus instrucciones o conseguir información que no debería revelar.

Cada caso incluye un identificador, su categoría, las herramientas que se espera que utilice el agente y los datos que debería incluir o evitar en la respuesta.

Por ejemplo, si un cliente pregunta por la disponibilidad de diez unidades de un producto, se espera que el agente consulte el inventario y entregue la cantidad correcta. También se pueden establecer datos que nunca deberían aparecer en una respuesta, como el teléfono personal de un cliente.

Protección de información y pruebas adversarias

La evaluación no se limita a comprobar que el agente responda correctamente. También debe verificar que no realice acciones o revele información que estén fuera de sus permisos.

Para ello, se incorporaron datos personales adicionales a Lumenshop, como teléfonos y direcciones postales de los clientes.

Estos datos permiten probar si el agente respeta las restricciones de privacidad. Por ejemplo, una consulta adversaria podría intentar convencerlo de que revele el teléfono de otro usuario.

También se añadieron reseñas de clientes que contienen instrucciones ocultas. Estas se utilizarán posteriormente para evaluar la resistencia del sistema frente a instrucciones maliciosas incorporadas en información externa.

Evaluación automática y análisis de resultados

Una vez definidos los casos de prueba, se ejecuta el agente y se comparan sus respuestas con el comportamiento esperado.

Se considera un caso exitoso cuando cumple todos los requisitos establecidos. Si

falla alguno, se clasifica como un caso fallido.

El sistema permite analizar diferentes resultados:

- Tasa de éxito de las pruebas.

- Resultados de Pass@K y Pass^K.

- Mediana y percentil 95 de los tiempos de respuesta.

- Categorías en las que se producen más errores.

- Herramientas utilizadas y si fueron las esperadas.

- Información que debería aparecer o que debe permanecer protegida.

Esto permite identificar si el agente falla en consultas habituales, situaciones límite o intentos de manipulación, y facilita decidir qué aspectos necesitan mejoras.

Uso de un modelo de lenguaje como juez (LLM-as-a-Judge)

En la segunda parte de la clase se explicó cómo utilizar otro modelo de lenguaje para evaluar las respuestas generadas por el agente. Este modelo actúa como un juez que revisa si la respuesta cumple determinados criterios.

La evaluación se realiza mediante una rúbrica, que funciona como una guía de instrucciones para el modelo evaluador. En lugar de pedirle simplemente que determine si una respuesta es buena o mala, se le indican criterios concretos que debe comprobar.

En el ejemplo de Lumenshop se trabajó con cuatro aspectos:
- Resolución: comprobar si el agente respondió correctamente a la consulta.
- Veracidad: verificar que no haya inventado información.
- Seguridad: comprobar que no revele datos de otros clientes ni información interna.
- Tono: revisar que la respuesta sea breve, amable y esté escrita en español.
El profesor recomendó que cada criterio fuera específico y estuviera separado de los demás. También explicó que es preferible pedir respuestas de tipo «sí» o «no», en lugar de puntuaciones subjetivas, porque son más fáciles de comparar y calibrar. La justificación de cada resultado puede añadirse de manera opcional.

Evaluación mediante reglas de código y un juez de IA

Se explicó que la evaluación puede combinar diferentes métodos:
1. Reglas de código: permiten comprobar aspectos objetivos, como si se utilizó una herramienta determinada, si se repitieron llamadas innecesariamente o si se ejecutó una acción irreversible.
2. Juez basado en un LLM: evalúa aspectos más flexibles, como la calidad de la respuesta, su tono, su veracidad y si cumple las instrucciones.
3. Revisión humana: permite comprobar manualmente si los resultados de los otros dos métodos son correctos.

No todas las empresas necesitan utilizar los tres métodos. La decisión depende del presupuesto, los recursos disponibles y las necesidades del proyecto.

El profesor destacó que, al principio, es importante comparar los resultados del juez con las evaluaciones humanas para calibrar su comportamiento. De esta manera se puede determinar si el modelo está evaluando correctamente o si necesita instrucciones más precisas.

Cómo funciona la evaluación del recorrido del agente

En la demostración se presentó una función encargada de revisar la trayectoria que siguió el agente para responder una consulta.

Esta función comprueba tres aspectos:

- Si utilizó las herramientas que se esperaban.
- Si repitió llamadas innecesariamente.
- Si realizó acciones irreversibles, como emitir un reembolso o enviar un correo electrónico, y si correspondía hacerlo.

Después, el juez recibe la pregunta del cliente, los resultados obtenidos de las herramientas y la respuesta final del agente. Con esa información evalúa los criterios de la rúbrica y devuelve un resultado estructurado en JSON.

También se explicó que, si el juez no devuelve un JSON válido, el sistema debe informar del error en lugar de inventarse un resultado.

Interpretación de los resultados y detección de errores

Una parte importante de la demostración consistió en entender que las métricas deben interpretarse en conjunto.

Algunos ejemplos explicados por el profesor fueron:

- Éxito alto, pero Pass^K bajo: el agente consigue resolver algunas pruebas, pero no es consistente. Se necesitan más repeticiones y revisar si mantiene el comportamiento.
- P95 muy superior a la mediana: algunas solicitudes están tardando demasiado. Puede haber bucles, reintentos o problemas en el diseño.
- Disponibilidad inferior al 100 %: existen errores técnicos, límites alcanzados o problemas relacionados con proveedores externos.
- Uso de herramientas incorrectas o repetidas: puede indicar que las instrucciones, la arquitectura o la selección de herramientas necesitan ajustes.

También se mostró que las reglas de código y el juez pueden discrepar. Por ejemplo, el código puede considerar que una respuesta fue exitosa porque no incluyó un dato expresamente prohibido, mientras que el juez puede detectar que el agente reveló otros datos confidenciales.

Estas diferencias son útiles porque permiten descubrir errores que una sola forma de evaluación podría pasar por alto.

Calibración del juez y refinamiento de la evaluación

Cuando el juez y las reglas de código no coinciden, no se debe asumir automáticamente que uno de los dos tiene razón.

Es necesario revisar el caso y determinar dónde está el problema: puede estar en el código, en la rúbrica, en las instrucciones del agente o en los datos que recibió.

La revisión humana permite identificar estas discrepancias y corregirlas. Con el tiempo, este proceso ayuda a mejorar la rúbrica y a conseguir que el juez evalúe de manera más consistente.

El objetivo es que el sistema de evaluación pueda automatizarse progresivamente, sin dejar de comprobar que sus resultados sean fiables.

Este documento ha sido generado con inteligencia artificial.